

Hardcoded Values Elimination

Complete Fix Reference

Every value in CardioReport must trace back to either (a) the raw data file, (b) a configuration parameter, or (c) a deterministic computation from (a) and (b). Nothing should be a magic string or magic number sitting in the middle of a module.

The rule: If you change a threshold in settings, every downstream value that depends on it must automatically change. If it does not, you have a hardcoded bug. This document fixes all of them.

1. The Single Source of Truth: Settings Object

Create one settings object that every module imports. No module should define its own thresholds, labels, or defaults. Here is the complete settings specification:

```
# settings.py (or config.py)

from pydantic import BaseModel

class ThresholdSettings(BaseModel):
    # ■■ Episode detection thresholds ■■
    bradycardia_hr_avg: float = 50.0 # avg_hr below this
    severe_brady_hr_min: float = 45.0 # min_hr below this
    tachycardia_hr_avg: float = 100.0 # avg_hr above this
    tachypnea_rr_avg: float = 24.0 # avg_rr above this
    low_confidence_cnt: int = 30 # cnt below this

    # ■■ Severity scoring weights ■■
    base_severe_brady: int = 5
    base_bradycardia: int = 3
    base_tachycardia: int = 3
    base_tachypnea: int = 2
    duration_bonus_per_hour: int = 1 # +1 per hour beyond first
    coupling_bonus: int = 2 # +2 if HR/RR co occur
    low_conf_penalty: int = 1 # -1 if low confidence

    # ■■ Triage rules ■■
    red_severe_brady_hours: int = 4 # severe brady >= N hours = RED
    red_tachypnea_hours: int = 8 # tachypnea >= N hours = RED
    red_coupled_severity: int = 9 # coupled + sev >= N = RED
    yellow_min_severity: int = 5 # max sev >= N = YELLOW

    # ■■ Trend assessment rules ■■
    progressive_min_severity: int = 9
    progressive_coupled_hours: int = 10
    intermittent_min_severity: int = 5
```

```
intermittent_min_hours: int = 5

# ■■ Severity band boundaries ■■
band_s1_min: int = 5
band_s2_min: int = 9
band_s3_min: int = 13

# ■■ Report generation ■■
max_events_table_rows: int = 8
max_action_bullets: int = 5
chart_dpi: int = 200
use_llm: bool = False

# Global singleton
settings = ThresholdSettings()
```

Every module does: `from settings import settings`
No module ever writes 50, 45, 100, or 24 as a literal number. Ever.

2. The Five Bugs You Found (and Their Fixes)

Bug 1: pdf_render.py line 236 — Hardcoded resolution string

What is wrong:

```
# BROKEN: hardcoded string ignores actual data resolution
Paragraph('Resolution: Hourly HR/RR', style)
```

How to compute it:

The data resolution describes how the raw data is aggregated. This should be computed from the actual data, not assumed. There are two approaches:

Approach A: Infer from the data itself

```
# In signal_engine.py
def compute_data_resolution(df):
    """Infer the time resolution from the data."""
    if len(df) < 2:
        return "Insufficient data"

    # Sort by datetime, compute median gap between consecutive rows
    sorted_dates = df['Date'].sort_values()
    gaps = sorted_dates.diff().dropna()
    median_gap = gaps.median()

    # Classify based on median gap
    hours = median_gap.total_seconds() / 3600

    if hours <= 0.25:
        return "15 minute aggregates (HR/RR min/avg/max)"
    elif hours <= 1.5:
        return "Hourly aggregates (HR/RR min/avg/max)"
    elif hours <= 6:
        return "Multi hour aggregates (HR/RR min/avg/max)"
    else:
        return "Daily aggregates (HR/RR min/avg/max)"
```

Approach B: Detect from column presence

```
# In signal_engine.py
def compute_data_resolution(df):
    """Determine resolution from available columns."""
    has_minmax = all(c in df.columns for c in ['min_hr', 'max_hr', 'min_rr', 'max_rr'])
    has_avg = all(c in df.columns for c in ['avg_hr', 'avg_rr'])

    if has_minmax and has_avg:
        # Check time gaps for hourly vs other
        sorted_dates = df['Date'].sort_values()
        median_gap_h = sorted_dates.diff().dropna().median().total_seconds() / 3600
        if median_gap_h <= 1.5:
            return "Hourly aggregates (HR/RR min/avg/max)"
        else:
            return f"{median_gap_h:.0f} hour aggregates (HR/RR min/avg/max)"
    elif has_avg:
        return "Average only (no min/max)"
    else:
```

```
return "Raw readings"
```

Where it flows:

```
# In models.py
class ReportData(BaseModel):
    data_resolution: str # NO DEFAULT. Must be computed.

# In the pipeline (main.py)
report.data_resolution = signal_engine.compute_data_resolution(wdf)

# In pdf_render.py
Paragraph(f'Resolution: {report.data_resolution}', style)
```

Bug 2: signal_engine.py — Trend label comparison mismatch

What is wrong:

```
# BROKEN: compares against partial strings
if trend == "Progressively unstable": # FAILS
if trend == "Escalating instability": # FAILS

# Because compute_trend() returns full labels:
# "Progressively unstable vital sign pattern"
# "Escalating instability toward end of monitoring"
```

The fix: Define labels as constants in settings

```
# In settings.py – add these as class constants
class TrendLabels:
    STABLE = "Stable vital sign pattern"
    INTERMITTENT = "Intermittently unstable vital sign pattern"
    PROGRESSIVE = "Progressively unstable vital sign pattern"
    ESCALATING = "Escalating instability toward end of monitoring"

class TriageLabels:
    GREEN = "Green"
    YELLOW = "Yellow"
    RED = "Red"

class ActionPostureLabels:
    ROUTINE = "Routine review"
    CLOSER = "Closer clinical observation warranted"
    PROVIDER = "Provider review advised"
    URGENT = "Urgent provider review advised"
```

Now every comparison uses the constant:

```
# In signal_engine.py (or wherever compute_action_posture lives)
from settings import TrendLabels, ActionPostureLabels

def compute_action_posture(episodes, trend_label):
    if not episodes:
        return ActionPostureLabels.ROUTINE

    mx = max(e['severity_score'] for e in episodes)
    cp = any(e.get('cooccur_brady_tachypnea', 0) for e in episodes)

    if mx >= settings.red_coupled_severity or cp:
        return ActionPostureLabels.PROVIDER
    if mx >= settings.yellow_min_severity:
```

```
return ActionPostureLabels.CLOSER  
return ActionPostureLabels.ROUTINE
```

The principle: No string literal that appears in more than one file should exist as a raw string. It must be a named constant imported from settings. If you ever rename a label, you change it in one place and every comparison updates.

Bug 3: models.py — data_resolution default value

What is wrong:

```
# BROKEN: default does not match actual data
class ReportData(BaseModel):
    data_resolution: str = "Hourly aggregates" # assumed, not computed
```

The fix:

Remove the default entirely. Force the pipeline to compute it. If data_resolution is not set before rendering, Pydantic will raise a validation error, which is exactly what you want. A missing value should fail loudly, not silently render an incorrect string.

```
# FIXED: no default forces computation
class ReportData(BaseModel):
    data_resolution: str # REQUIRED. Computed by signal_engine.
    trend_assessment: str # REQUIRED. Computed by episodes.
    triage_color: str # REQUIRED. Computed by episodes.
    action_posture: str # REQUIRED. Computed by episodes.
    narrative: str # REQUIRED. Computed by narrative_ai.
    actions: list[str] # REQUIRED. Computed by narrative_ai.
    # ... stats, episodes, quality, etc.
```

Rule for models.py: Only fields that are truly optional (like patient_baseline_hr, which may not be available for all patients) should have defaults. Every field that the pipeline MUST compute should have no default, so missing computation causes an immediate error, not a silent wrong value on the report.

Bug 4: models.py — trend_assessment default

What is wrong:

```
# BROKEN: default is a partial label
trend_assessment: str = "Stable" # should be "Stable vital sign pattern"
```

The fix:

Same solution as Bug 3: remove the default. And use the constant from settings everywhere:

```
# In episodes.py
from settings import settings, TrendLabels

def compute_trend(episodes):
    if not episodes:
        return TrendLabels.STABLE

    mx = max(e['severity_score'] for e in episodes)
    cp = any(e.get('cooccur_brady_tachypnea', 0) for e in episodes)
    tot = sum(e['hours'] for e in episodes)

    if mx >= settings.progressive_min_severity or \
       (cp and tot > settings.progressive_coupled_hours):
        return TrendLabels.PROGRESSIVE

    if mx >= settings.intermittent_min_severity or \
       tot > settings.intermittent_min_hours:
        return TrendLabels.INTERMITTENT

    return TrendLabels.STABLE
```

Now if you ever want to add the fourth label (ESCALATING), you add it to TrendLabels, add a rule in compute_trend(), and every comparison across all modules picks it up automatically.

Bug 5: charts.py — Hardcoded threshold lines

What is wrong:

```
# BROKEN: chart draws lines at fixed values
ax1.axhline(y=50, ...) # should be settings.bradycardia_hr_avg
ax1.axhline(y=100, ...) # should be settings.tachycardia_hr_avg
ax2.axhline(y=24, ...) # should be settings.tachypnea_rr_avg

# Same in histogram:
ax1.axvline(50, ...) # hardcoded
ax1.axvline(100, ...) # hardcoded
ax2.axvline(24, ...) # hardcoded
```

The fix: Pass settings into chart functions

```
# In charts.py
from settings import settings

def chart_candlestick(daily_df, episodes):
    fig, (ax1, ax2) = plt.subplots(2, 1, ...)

    # ... draw bars ...

    # HR thresholds from settings
    ax1.axhline(y=settings.bradycardia_hr_avg,
                color='#C0392B', linewidth=0.6, linestyle='--', alpha=0.4)
    ax1.axhline(y=settings.tachycardia_hr_avg,
                color='#C0392B', linewidth=0.6, linestyle='--', alpha=0.4)
    ax1.text(len(x)-0.5, settings.bradycardia_hr_avg,
            f' {settings.bradycardia_hr_avg:.0f} bpm',
            fontsize=5.5, color='#C0392B', alpha=0.6, va='bottom')
    ax1.text(len(x)-0.5, settings.tachycardia_hr_avg,
            f' {settings.tachycardia_hr_avg:.0f} bpm',
            fontsize=5.5, color='#C0392B', alpha=0.6, va='bottom')

    # RR threshold from settings
    ax2.axhline(y=settings.tachypnea_rr_avg,
                color='#E65100', linewidth=0.6, linestyle='--', alpha=0.4)
    ax2.text(len(x)-0.5, settings.tachypnea_rr_avg,
            f' {settings.tachypnea_rr_avg:.0f} brpm',
            fontsize=5.5, color='#E65100', alpha=0.6, va='bottom')

    return fig_to_buffer(fig)

def chart_histogram(wdf):
    fig, (ax1, ax2) = plt.subplots(1, 2, ...)

    # HR histogram thresholds from settings
    ax1.axvline(settings.bradycardia_hr_avg,
                color='#C0392B', linewidth=0.8, linestyle=':', alpha=0.5)
    ax1.axvline(settings.tachycardia_hr_avg,
                color='#C0392B', linewidth=0.8, linestyle=':', alpha=0.5)

    # RR histogram threshold from settings
    ax2.axvline(settings.tachypnea_rr_avg,
                color='#E65100', linewidth=0.8, linestyle=':', alpha=0.5)
```

```
return fig_to_buffer(fig)
```

Now when you change a threshold: If you set `bradycardia_hr_avg` to 48 instead of 50, the episode detector uses 48, the severity scoring uses 48, the triage rules reference 48, the candlestick chart draws the dashed line at 48, the histogram marks 48, and the chart labels say "48 bpm". One change, zero manual updates.

3. Complete Value Flow Map

Every value on the report, where it is computed, and what it depends on:

Report Value	Computed By	Depends On	Hardcoded?
Patient ID	excel_ingest.py	Sheet name in Excel file	No
Period label	API request	start_date, end_date from frontend	No
Report date	Pipeline	datetime.now() at generation time	No
Data resolution	signal_engine.py	Median time gap between rows in windowed data	MUST FIX
Disclaimer text	settings.py	Constant string (OK to hardcode; it is a legal text)	OK
Triage badge	episodes.py	episodes list + settings.red_* and settings.yellow_*	No
Trend assessment	episodes.py	episodes list + settings.progressive_* and settings.intermittent_*	MUST FIX
Action posture	episodes.py	episodes list + settings thresholds	MUST FIX
Narrative text	narrative_ai.py	episodes + stats + quality + settings.use_llm	No
Action bullets	narrative_ai.py	Episode condition types + coupling flag	No
Events table rows	episodes.py	Hourly data + settings thresholds	No
Severity scores	episodes.py	Episode type + duration + coupling + settings.base_* and bonus_*	No
Severity bands	episodes.py	Score + settings.band_s1_min, s2_min, s3_min	No
Comment phrases	episodes.py	Severity band lookup (BAND_CMT dict)	No
Stats (Mean/Min/Max/P5/P95)	signal_engine.py	Windowed hourly data, pandas aggregations	No
Coverage %	signal_engine.py	Row count / expected hours from date range	No
Low confidence count	signal_engine.py	Count of rows where cnt < settings.low_confidence_cnt	No
Candlestick chart	charts.py	Daily aggregates + episodes + settings thresholds for ref lines	MUST FIX
Histogram chart	charts.py	Hourly avg_hr/avg_rr + settings thresholds for ref lines	MUST FIX
Chart threshold labels	charts.py	settings threshold values formatted as f string	MUST FIX

4. Episode Detection: Complete Computation Reference

Since you asked how everything is computed, here is the full episode detection algorithm with no hardcoded values. Every threshold, weight, and boundary comes from settings.

Step 1: Flag each hour

```
def flag_hours(df):
    """Flag each hour against all thresholds. Returns dict of boolean Series."""
    return {
        'Bradycardia': df['avg_hr'] < settings.bradycardia_hr_avg,
        'Severe bradycardia': df['min_hr'] < settings.severe_brady_hr_min,
        'Tachycardia': df['avg_hr'] > settings.tachycardia_hr_avg,
        'Tachypnea': df['avg_rr'] > settings.tachypnea_rr_avg,
    }
```

Step 2: Group consecutive flagged hours into episodes

```
def group_episodes(df, flags, condition_name):
    """Group consecutive True hours into episode dicts."""
    flagged = df[flags].copy()
    if flagged.empty:
        return []

    episodes = []
    flagged = flagged.sort_values('Date')

    # Detect breaks: where time gap > 1 hour
    flagged['gap'] = flagged['Date'].diff() > pd.Timedelta(hours=1)
    flagged['group'] = flagged['gap'].cumsum()

    for _, grp in flagged.groupby('group'):
        ep = {
            'condition': condition_name,
            'start': grp['Date'].min().isoformat(),
            'end': grp['Date'].max().isoformat(),
            'hours': len(grp),
            'hr_avg_mean': round(grp['avg_hr'].mean(), 1),
            'hr_min_min': round(grp['min_hr'].min(), 1),
            'hr_max_max': round(grp['max_hr'].max(), 1),
            'rr_avg_mean': round(grp['avg_rr'].mean(), 1),
            'rr_max_max': round(grp['max_rr'].max(), 1),
            'low_cnt_hours': int((grp['cnt'] < settings.low_confidence_cnt).sum()),
        }

        # Co occurrence check
        if condition_name in ('Bradycardia', 'Severe bradycardia'):
            ep['cooccur_brady_tachypnea'] = int(
                (grp['avg_rr'] > settings.tachypnea_rr_avg).any()
            )
        else:
            ep['cooccur_brady_tachypnea'] = 0

        episodes.append(ep)

    return episodes
```

Step 3: Compute severity score

```
BASE_SCORES = {
    'Severe bradycardia': settings.base_severe_brady, # 5
    'Bradycardia': settings.base_bradycardia, # 3
    'Tachycardia': settings.base_tachycardia, # 3
    'Tachypnea': settings.base_tachypnea, # 2
}

def compute_severity(episode):
    base = BASE_SCORES.get(episode['condition'], 0)

    # Duration: +1 per hour beyond the first
    duration_bonus = (episode['hours'] - 1) * settings.duration_bonus_per_hour

    # Coupling: +2 if HR and RR co occur
    coupling = settings.coupling_bonus if episode.get('cooccur_brady_tachypnea') else 0

    # Low confidence: -1 if any hour has low sample count
    penalty = settings.low_conf_penalty if episode.get('low_cnt_hours', 0) > 0 else 0

    score = base + duration_bonus + coupling - penalty
    return max(score, 0) # floor at 0
```

Step 4: Assign severity band

```
def severity_band(score):
    if score >= settings.band_s3_min: return 'S3'
    if score >= settings.band_s2_min: return 'S2'
    if score >= settings.band_s1_min: return 'S1'
    return 'S0'
```

Step 5: Compute triage (no hardcoded numbers)

```
def compute_triage(episodes):
    if not episodes:
        return TriageLabels.GREEN

    mx = max(e['severity_score'] for e in episodes)
    cp = any(e.get('cooccur_brady_tachypnea', 0) for e in episodes)

    sev_long = any(
        e['hours'] >= settings.red_severe_brady_hours
        and 'Severe' in e['condition']
        for e in episodes
    )
    tr_long = any(
        e['hours'] >= settings.red_tachypnea_hours
        and e['condition'] == 'Tachypnea'
        for e in episodes
    )

    if sev_long or tr_long or (cp and mx >= settings.red_coupled_severity):
        return TriageLabels.RED
    if mx >= settings.yellow_min_severity:
        return TriageLabels.YELLOW
    return TriageLabels.GREEN
```

Step 6: Compute trend (no hardcoded numbers)

```
def compute_trend(episodes):
    if not episodes:
        return TrendLabels.STABLE

    mx = max(e['severity_score'] for e in episodes)
    cp = any(e.get('cooccur_brady_tachypnea', 0) for e in episodes)
    tot = sum(e['hours'] for e in episodes)

    if mx >= settings.progressive_min_severity or \
        (cp and tot > settings.progressive_coupled_hours):
        return TrendLabels.PROGRESSIVE

    if mx >= settings.intermittent_min_severity or \
        tot > settings.intermittent_min_hours:
        return TrendLabels.INTERMITTENT

    return TrendLabels.STABLE
```

5. Verification Checklist

After applying all fixes, run this checklist to verify no hardcoded values remain:

Module	Verification
settings.py	All thresholds, weights, labels, and caps defined as named fields
excel_ingest.py	No hardcoded patient IDs or column names (already clean per your audit)
signal_engine.py	compute_data_resolution() infers from data, not hardcoded string
signal_engine.py	Low confidence uses settings.low_confidence_cnt, not literal 30
episodes.py	All threshold comparisons use settings.bradycardia_hr_avg etc.
episodes.py	Severity formula uses settings.base_*, settings.duration_bonus_per_hour, settings.coupling_bonus
episodes.py	Band boundaries use settings.band_s1_min, s2_min, s3_min
episodes.py	compute_triage uses settings.red_* and settings.yellow_* thresholds
episodes.py	compute_trend uses settings.progressive_* and settings.intermittent_*
episodes.py	compute_action_posture compares against TrendLabels.* constants, not partial strings
narrative_ai.py	Plain language mapping uses PLAIN_LANG dict, not inline strings
narrative_ai.py	LLM prompt includes threshold values from settings (so AI knows the thresholds)
charts.py	Candlestick reference lines use settings.bradycardia_hr_avg, tachycardia_hr_avg, tachypnea_rr_avg
charts.py	Histogram threshold markers use same settings values
charts.py	Chart labels format threshold values from settings (f string, not literal)
charts.py	Chart DPI uses settings.chart_dpi
pdf_render.py	Resolution string uses report.data_resolution, not hardcoded
pdf_render.py	Events table capped at settings.max_events_table_rows
pdf_render.py	Triage badge text uses TriageLabels.* for comparison
models.py	No defaults on data_resolution, trend_assessment, triage_color, action_posture

The ultimate test:

Change settings.bradycardia_hr_avg from 50 to 48. Run the pipeline on the same data. Verify that ALL of the following change automatically with zero code edits:

- Episode detection flags fewer hours (some hours with avg_hr between 48 and 50 no longer flagged)
- Episode count decreases or episode durations shorten
- Severity scores recalculate for affected episodes
- Triage color may change (if the change eliminates a RED trigger)
- Trend assessment may change
- Narrative text changes (different episode counts and types)
- Candlestick chart draws the dashed line at 48, not 50
- Candlestick chart label says "48 bpm", not "50 bpm"
- Histogram threshold marker moves to 48
- Events table rows change (different episodes, different sort order)

If all of these update from a single settings change with zero code edits, your pipeline has zero hardcoded values. That is the goal.